

Lab 2: Multilayer Perceptron (MLP)

OBJECTIVE: Implement a Backpropagation Algorithm from scratch. (Assume you have AND/XOR gate with two inputs, train a multilayer perceptron).

THEORY:

Multilayer Perceptron (MLP):

- A neural network with one or more hidden layers between input and output.
- Each neuron applies a weighted sum followed by a non-linear activation function (e.g., sigmoid).

Backpropagation Algorithm:

- Used to train MLPs by minimizing the error between predicted and actual outputs.
- Steps:
 - Forward Pass: Compute outputs layer by layer.
 - Error Calculation: Compute difference between predicted and target output.
 - Backward Pass: Compute gradients of the error w.r.t. each weight using chain rule.
 - Weight Update: Adjust weights in the opposite direction of the gradient using a learning rate

Importance: Backpropagation allows MLPs to learn non-linear patterns (e.g., XOR), which cannot be solved by single-layer perceptrons.

ALGORITHM (BACKPROPAGATION):

- Initialize weights and biases randomly for hidden and output layers
- For each epoch:
 - Forward Pass:
 - Compute outputs of hidden layer neurons
 - Compute output layer neuron output
 - Error Calculation:
 - Compute difference between predicted and target output
 - Backward Pass:
 - Compute gradients for output layer
 - Compute gradients for hidden layer
- Update Weights and Biases using learning rate

- Repeat until error is minimized

SOURCE CODE:

CELL 1: IMPORT LIBRARIES

```
In [1]: import numpy as np
```

CELL 2: ACTIVATION FUNCTIONS

```
In [2]: # Sigmoid activation and its derivative
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)
```

CELL 3: DATA (AND/XOR GATES)

```
In [4]: # Input for AND/XOR
X = np.array([[0,0],[0,1],[1,0],[1,1]])

# Target output: change y to AND or XOR
y_and = np.array([[0],[0],[0],[1]])
y_xor = np.array([[0],[1],[1],[0]])
```

CELL 4: INITIALIZE WEIGHTS AND BIASES

```
In [5]: # Seed for reproducibility
np.random.seed(42)

# 2 inputs -> 2 hidden neurons -> 1 output
input_neurons = 2
hidden_neurons = 2
output_neurons = 1

# Weights
wh = np.random.uniform(size=(input_neurons, hidden_neurons))
bh = np.random.uniform(size=(1, hidden_neurons))
wo = np.random.uniform(size=(hidden_neurons, output_neurons))
bo = np.random.uniform(size=(1, output_neurons))

# Learning rate
lr = 0.5
```

CELL 5: TRAINING FUNCTION

```
In [6]: def train_mlp(X, y, wh, bh, wo, bo, lr, epochs=10000):
    for epoch in range(epochs):
        # Forward pass
        hidden_input = np.dot(X, wh) + bh
        hidden_output = sigmoid(hidden_input)
        final_input = np.dot(hidden_output, wo) + bo
        final_output = sigmoid(final_input)

        # Error
```

```

    error = y - final_output

    # Backward pass
    d_output = error * sigmoid_derivative(final_output)
    error_hidden = d_output.dot(wo.T)
    d_hidden = error_hidden * sigmoid_derivative(hidden_output)

    # Update weights and biases
    wo += hidden_output.T.dot(d_output) * lr
    bo += np.sum(d_output, axis=0, keepdims=True) * lr
    wh += X.T.dot(d_hidden) * lr
    bh += np.sum(d_hidden, axis=0, keepdims=True) * lr

    return wh, bh, wo, bo

```

CELL 6: TRAIN ON XOR

```

In [7]: # Train MLP on XOR
wh, bh, wo, bo = train_mlp(X, y_xor, wh, bh, wo, bo, lr, epochs=10000)

# Test
hidden_output = sigmoid(np.dot(X, wh) + bh)
final_output = sigmoid(np.dot(hidden_output, wo) + bo)
print("Predicted XOR outputs:")
print(np.round(final_output))

```

Predicted XOR outputs:

```

[[0.]
 [1.]
 [1.]
 [0.]]

```

RESULTS:

- The multilayer perceptron successfully learned the XOR gate, producing outputs [[0.], [1.], [1.], [0.]] (equivalent to [0.0, 1.0, 1.0, 0.0]).
- Hidden layer neurons allowed the network to capture non-linear patterns, unlike a single-layer perceptron.
- The network can also correctly learn AND/OR gates with simple linear outputs.

CONCLUSION:

- MLPs with backpropagation can learn non-linear functions like XOR.
- Hidden layers are essential for solving problems that are not linearly separable.
- Backpropagation efficiently updates weights and biases to minimize error, demonstrating how neural networks learn from data.